

Day 117-Part-2: MLflow Fundamentals — Beginner Experiment Tracking

1. Day number

Day 117-Part-2

2. Topic name

MLflow Fundamentals and Experiment Tracking

MLflow helps you **record, organize, and compare machine-learning experiments**.

Its current Tracking system revolves around **experiments, runs**, parameters, metrics, models, and artifacts. :chatgpt-content-reference{index="0"}

3. Connection

Yesterday you learned an end-to-end scikit-learn workflow:

```
Dataset
  ↓
Preprocessing
  ↓
Model
  ↓
fit()
  ↓
predict()
  ↓
Evaluation
```

Now imagine you train the model several times:

Run 1 → Logistic Regression → accuracy 0.81

Run 2 → Decision Tree → accuracy 0.84

Run 3 → another setting → accuracy 0.82

After many experiments, you may start wondering:

Which settings did I use for Run 2?
Which model produced the best metric?
Where did I save that trained model?

This is where **MLflow** becomes useful.

Conceptually:

```
Machine-learning experiments
  ↓
```

MLflow



Record what you tried
and what happened

4. Important topics

For your first MLflow lesson, focus on:

- experiment
 - run
 - parameter
 - metric
 - artifact
 - model
 - MLflow Tracking
 - MLflow UI
 - `start_run()`
 - logging
 - autologging
-

5. What problem does MLflow solve?

Suppose you are trying different models.

You manually write:

```
Experiment 1  
model = Logistic Regression  
accuracy = 82%
```

```
Experiment 2  
model = Decision Tree  
accuracy = 85%
```

```
Experiment 3  
model = Random Forest  
accuracy = 87%
```

That works for three experiments.

But imagine:

100 experiments

Now you need to remember:

Which model?

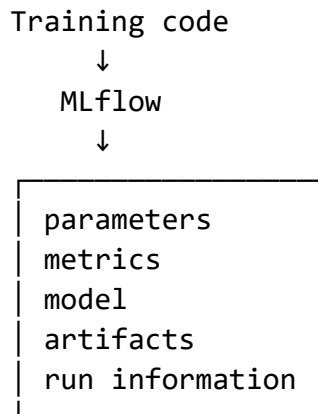
Which settings?

Which accuracy?
Which dataset?
Which output files?
Which trained model?

Keeping this manually in notebooks becomes inconvenient.

MLflow Tracking provides APIs and a UI for recording parameters, metrics, output files, models, and other run information. :chatgpt-content-reference{index="1"}

A simple mental model is:



6. Foundational concepts

Experiment

An **experiment** is a container that groups related ML runs.

Imagine you are working on:

Customer Purchase Prediction

That could be one MLflow experiment.

Inside it:

Customer Purchase Prediction

```
|— Run 1
|— Run 2
|— Run 3
|— Run 4
```

MLflow describes experiments as groups of runs and models associated with a particular task. :chatgpt-content-reference{index="2"}

Run

A **run** is one execution of your machine-learning training process.

For example:

Run 1

Model:

Logistic Regression

Training:

one complete execution

Accuracy:

0.83

Change something and train again:

Run 2

Model:

Logistic Regression

different setting

Accuracy:

0.86

That is a different run.

So remember:

Experiment

↓

contains many

↓

Runs

7. Parameter

A **parameter** is usually a setting used for your experiment or model.

For example:

`max_depth = 5`

for a decision tree.

Or:

`C = 1.0`

for Logistic Regression.

Conceptually:

Parameter

=

"What setting did I use?"

You can log such information with MLflow. The Tracking API includes functions such as `mlflow.log_param()`. :chatgpt-content-reference{index="3"}

8. Metric

A **metric** tells you something about model performance.

Examples:

`accuracy = 0.87`

`F1 = 0.84`

`RMSE = 4.3`

Conceptually:

Metric

=

"How did the model perform?"

MLflow provides functions such as:

`mlflow.log_metric(...)`

for recording metrics. :chatgpt-content-reference{index="4"}

9. Parameter vs metric

This distinction is important.

Suppose:

Decision Tree

`max_depth = 5`

`accuracy = 0.86`

Then:

`max_depth = 5`

is a **parameter** because you chose that setting.

But:

`accuracy = 0.86`

is a **metric** because it describes the result.

Remember:

Parameter
→ what I chose

Metric
→ what I got

10. Artifact

An **artifact** is an output file produced or saved during an experiment.

Examples could include:

trained model file

chart

prediction CSV

image

report

For example:

Run 5

Parameters:
max_depth = 4

Metrics:
accuracy = 0.88

Artifacts:
confusion_matrix.png
trained_model

MLflow's artifact storage is designed for output files such as model files, images, and data files. :chatgpt-content-reference{index="5"}

11. Easy example

Imagine you trained three decision trees.

Run 1

max_depth = 2
accuracy = 0.78

Run 2

max_depth = 4
accuracy = 0.84

Run 3

max_depth = 6
accuracy = 0.82

Instead of keeping this information manually, MLflow can organize it roughly like:

Run	Parameter: max_depth	Metric: accuracy
Run 1	2	0.78
Run 2	4	0.84
Run 3	6	0.82

Now it becomes much easier to answer:

| What did I try?

and:

| What result did each run produce?

This is called **experiment tracking**.

12. Basic MLflow workflow

A beginner workflow looks like:

Create/select experiment

↓

Start run

↓

Train model

↓

Log parameters

↓

Evaluate model

↓

Log metrics

↓

Optionally log model/artifacts

↓

End run

↓

Inspect results in MLflow

13. start_run() idea

MLflow can explicitly start a tracking run using:

```
mlflow.start_run()
```

The official documentation shows this pattern for creating a run and then logging parameters and metrics inside it. :chatgpt-content-reference{index="6"}

Conceptually:

START RUN

```
    train model
```

```
    save parameter information
```

```
    save metric information
```

END RUN

Think of it as opening a folder for one experiment attempt.

Everything you record during that run belongs together.

14. Beginner-friendly conceptual pseudocode

START

```
load dataset
```

```
split into:
```

```
    training data
```

```
    testing data
```

```
select MLflow experiment
```

```
start MLflow run
```

```
    choose model settings
```

```
    log model settings as parameters
```

```
    train model
```

```
    make predictions
```

```
    calculate accuracy
```

```
    log accuracy as a metric
```

optionally save the trained model

end MLflow run

open MLflow interface

inspect the run

END

Notice that MLflow does **not replace** scikit-learn.

You still do:

`model.fit()`

`model.predict()`

calculate metric

MLflow sits around that process and **tracks what happened**.

15. How MLflow fits with yesterday's Pipeline

Yesterday:

Dataset



Pipeline



Preprocessing



Model



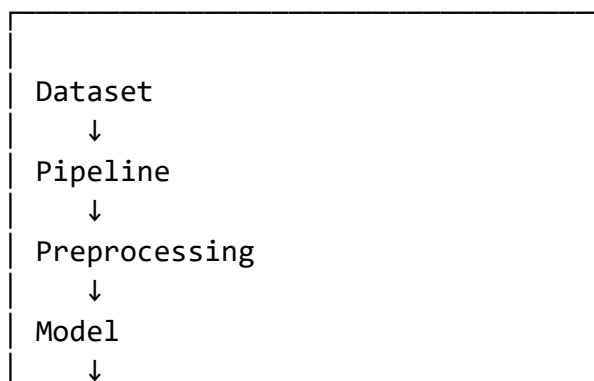
Prediction

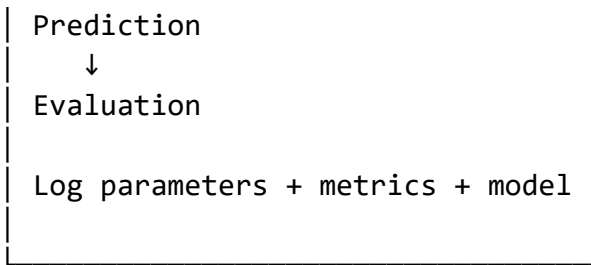


Evaluation

Today:

MLflow Run





So:

scikit-learn
→ builds/trains the ML model

MLflow
→ tracks the experiment

16. MLflow UI

MLflow provides a UI where you can inspect experiments and their runs.

The current MLflow Tracking UI supports viewing and comparing runs, searching by parameters or metrics, visualizing metrics, and inspecting artifacts. :chatgpt-content-reference{index="7"}

Think of it as a dashboard:

Experiment: Customer Prediction

Run	Model	Accuracy

Run 1	Logistic	0.80
Run 2	Tree	0.84
Run 3	Logistic	0.83

Clicking a run lets you inspect information associated with that run.

For a basic local setup, the current quickstart shows starting the MLflow server with:

```
mlflow server --port 5000
```

and then opening the local MLflow interface in your browser. :chatgpt-content-reference{index="8"}

You don't need to configure remote servers or cloud storage for today's lesson.

17. Manual logging

One way to use MLflow is to explicitly tell it what to record.

Conceptually:

```
start run
```

```
log:
    model type
```

```
log:
    model parameters
```

```
train model
```

```
calculate:
    accuracy
```

```
log:
    accuracy
```

```
end run
```

The important MLflow ideas are:

```
mlflow.start_run()
```

```
mlflow.log_param(...)
```

```
mlflow.log_metric(...)
```

Don't worry about memorizing syntax yet.

18. Autologging

MLflow can also automatically capture information from supported ML libraries.

For scikit-learn, the official quickstart demonstrates:

```
mlflow.sklearn.autolog()
```

before ordinary model training. MLflow can then capture relevant parameters, metrics, model information, and metadata automatically. :chatgpt-content-reference{index="9"}

Conceptually:

Without autologging:

```
train
↓
manually log parameter
↓
manually log metric
↓
manually log model
```

versus:

With autologging:

```
enable autologging
```

↓

```
train model normally
```

↓

```
MLflow records supported information
```

For learning, it is useful to understand **manual logging first**, because it shows what MLflow is actually recording.

19. Problem statement

Use the same tiny classification idea from Day 117-Part-2.

Imagine this dataset:

Age	Income	City	Purchased
22	28000	Delhi	No
35	52000	Mumbai	Yes
29	41000	Delhi	No
44	75000	Bengaluru	Yes
38	61000	Mumbai	Yes

You already know how to conceptually build:

```
preprocessing
```

↓

```
Logistic Regression
```

Now add MLflow experiment tracking.

Design two runs.

Run 1

Model:

```
Logistic Regression
```

Parameter:

```
some chosen model setting
```

Metric:

```
accuracy
```

Run 2

Change one setting:

Parameter:
different value

Metric:
new accuracy

Your task is to identify:

1. What should the MLflow **experiment** represent?
2. What counts as one **run**?
3. Which value is a **parameter**?
4. Which value is a **metric**?
5. What model or output could be stored as an **artifact/model**?
6. Why is comparing the two runs useful?

Do not build a large project.

20. Concepts used

This exercise combines:

Machine-learning workflow

+

Scikit-learn

+

Pipeline

+

Training

+

Evaluation

+

MLflow Tracking

And introduces:

Experiment

Run

Parameter

Metric

Artifact

Model tracking

MLflow UI

21. Thought process

When using MLflow, ask yourself five simple questions.

1. What project am I experimenting on?

For example:

Customer Purchase Prediction

That can become the **experiment**.

2. What am I trying this time?

For example:

Logistic Regression with $C = 1$

That is one **run**.

3. What settings did I choose?

$C = 1$

Those are **parameters**.

4. What result did I get?

$\text{accuracy} = 0.84$

Those are **metrics**.

5. What useful output should I save?

Perhaps:

trained model

or:

confusion_matrix.png

Those can be stored as model outputs/artifacts.

22. Easy edge cases

Edge case 1: Forgetting to log a parameter

Suppose:

Run 1 $\rightarrow$ $\text{accuracy} = 0.82$

Run 2 $\rightarrow$ $\text{accuracy} = 0.87$

but you forgot to record what changed.

Now the better metric is much less useful because you cannot easily reproduce why it happened.

This is exactly the kind of problem experiment tracking helps prevent.

Edge case 2: Logging training accuracy instead of test accuracy

Suppose you record:

$\text{accuracy} = 99\%$

but it came from your training data.

That may give an overly optimistic impression of generalization.

MLflow records what **you tell it to record**; it does not automatically make every evaluation choice correct.

Your ML understanding still matters.

Edge case 3: Too many meaningless runs

Running the exact same experiment repeatedly without meaningful changes can create clutter.

Experiment tracking works best when you know:

What changed?

Why did I run this?

What result am I comparing?

Edge case 4: MLflow does not improve the model automatically

This is important.

MLflow does not mean:

bad model

↓

MLflow

↓

great model

Instead:

model experiments

↓

MLflow

↓

organized experiment history

MLflow helps you **manage and understand your experiments**.

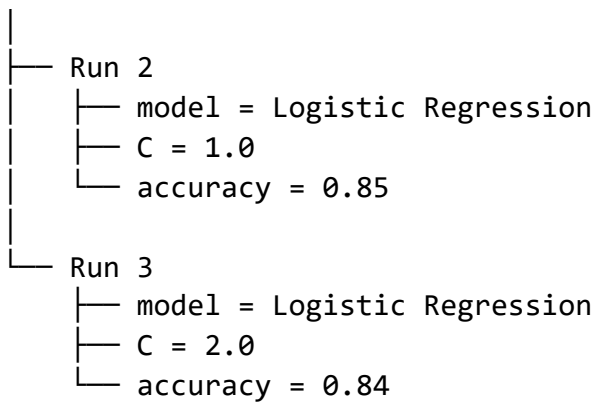
23. Expected conceptual result

After today's lesson, you should be able to understand something like:

Experiment:

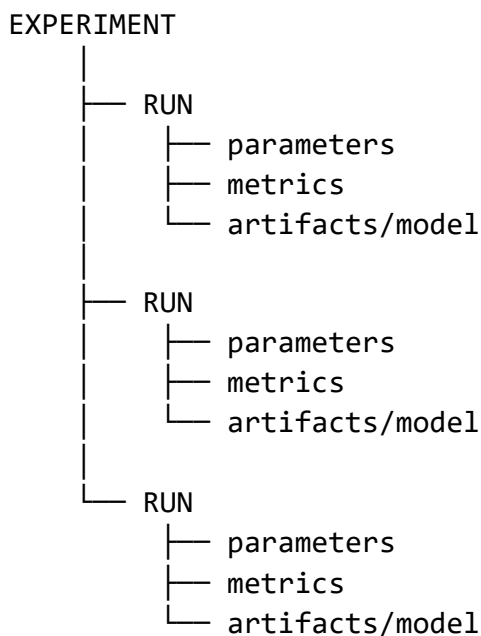
Customer Purchase Prediction

```
|— Run 1
|   |— model = Logistic Regression
|   |— C = 0.5
|   |— accuracy = 0.82
```



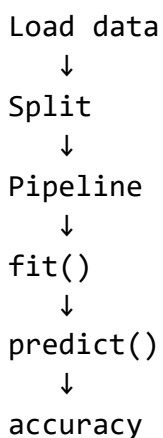
Instead of having three disconnected notebook executions, you now have an organized history of what you tried.

The core mental model is:



24. Hint only

Start from yesterday's workflow:



Now imagine putting an MLflow wrapper around one training attempt:

START MLFLOW RUN

```
choose model setting
  ↓
log parameter
  ↓
train model
  ↓
predict
  ↓
calculate accuracy
  ↓
log metric
  ↓
optionally log model
```

END MLFLOW RUN

Remember these four lines:

Experiment = collection of related attempts

Run = one attempt

Parameter = what you chose

Metric = what you got

And the big-picture progression from the last two days is:

Day 117-Part-2

How do I build a repeatable ML workflow?

↓

Day 117-Part-2

How do I keep track of all the experiments

I perform with that workflow?

That is the beginner-level purpose of **MLflow**. :chatgpt-content-reference{index="10"}